TASK COMPLETION REPORT

Task Ref. No: Chakra/IPD/Zozo/Actuation/Simulation

Task Description (2-3 lines): Imported an Onshape-exported 6-DOF biped URDF into MuJoCo, validated its structure, generated collision geometry, added six velocity-servo actuators, automatic ground clearance, an IMU sensor suite, and real-time telemetry.

SL no	Requirements	Taskbox
1.	CAED design of a basic 6-DOF leg &Environment Initialization	✓
2.	Physics fundamentals: gravity, timestep, contact parameters	✓
3.	URDF import and mesh-path resolution	✓
4.	URDF structural validation (link/joint tree, single connectivity)	✓
5.	Collision geometry generation and contact masking	✓
6.	Six-joint actuator integration (velocity servos)	✓
7.	Floating base and automatic ground-clearance placement	✓
8.	Physics engine configuration and stability debugging	
9.	IMU / MPU-9250-style sensor suite integration	✓
10.	Telemetry, diagnostics, and verification reporting	✓

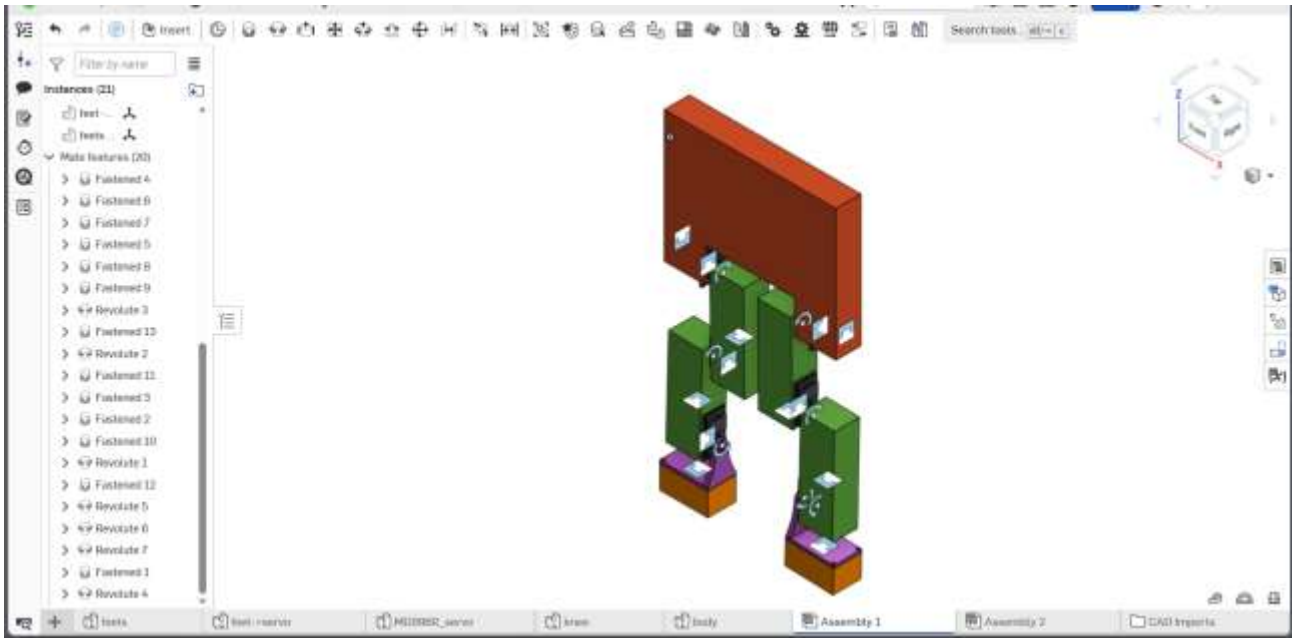
Task Completed (on/by): 10/11/26 – Krishna H S

Please see the following pages for a detailed breakdown of each requirement.

REQUIREMENTS BREAKDOWN

The task specification was the 100-exercise "MuJoCo Mastery Workbook" (Levels 0 to 6 and final projects). The first milestone was to load the robot, verify its mechanics, and attempt standing. The robot is around 30 cm MicroDuck-inspired biped with three joints per leg, exported from Onshape as assembly_1.urdf. Standing has not yet been achieved, and this report states that gap explicitly.

Requirement 1: CAED design of a basic 6-DOF leg &Environment Initialization



The CAED model was designed in Onshape to streamline the process. The motors which was used here were MG996R. These are assumed because I needed a starting point for the simulation.

- **What was required:** A reproducible Python and MuJoCo workspace in which the robot model, scene, meshes, and scripts are organized and can be launched reliably.
- **What was implemented:** For this I used a portable WinPython installation on a USB drive (`E:\chakra\WPy64-313180\python\python.exe`) with the MuJoCo Python bindings. The project folder contains `assembly_1.urdf`, `meshes/`, `scene.xml`, `diagnose.py`, `view_robot.py`, `chatgptsolution.py`, and `finialsolution.py`.
- **Evidence:** `chatgptsolution.py` raises `FileNotFoundError` if `scene.xml` or `assembly_1.urdf` is missing. `diagnose.py` prints the working directory and the contents of the `meshes` folder as its first two lines.
- **Remarks:** The toolchain is portable and needs no administrator installation. Script names preserve the original spelling (`finialsolution.py`) to match the submitted files.

Requirement 2: Physics fundamentals: gravity, timestep, contact parameters

- **What was required:** Correct configuration of gravity, simulation timestep, and floor contact parameters (softness, impedance, friction).
- **What was implemented:** `scene.xml` defines the floor plane (named `floor`), a checkerboard texture, a directional light, `timestep=0.002`, and `integrator=RK4`. `chatgptsolution.py` re-applies the timestep through `scene_spec.option.timestep` and sets gravity to `[0, 0, -9.81]`. Floor contact is tuned as `solref = [0.02, 1.0]`, `solimp = [0.90, 0.95, 0.001, 0.5, 2.0]`, and `friction = [1.0, 0.005, 0.0001]`.
- **Evidence:** The diagnostics block prints `Timestep: 0.002`. The floor lookup raises an error if no geometry named `floor` exists.
- **Status:** ✓ Complete
- **Remarks:** The scene file specifies RK4, while the main pipeline overrides this at runtime (see Requirement 8).

Requirement 3: URDF import and mesh-path resolution

- **What was required:** Load an Onshape-exported URDF whose mesh references use the ROS `package://` scheme.
- **What was implemented:** `diagnose.py` runs four tests: (A) the native C++ loader `MjModel.from_xml_path`, (B) the `MjSpec.from_file` Python loader with compilation, (C) absolute-path replacement followed by `from_xml_string`, and (D) a single STL loaded through inline MJCF. The function `strip_package_uri()` in `chatgptsolution.py` converts `package://assembly_1/meshes/body.stl` into `meshes/body.stl` for every `<mesh>` element.
- **Evidence:** The URDF references six unique STL files: `body`, `feet____servo`, `feets`, `kneew`, `servo_body`, and `shaft`. The patched URDF compiles through `MjSpec.from_string` and the diagnostics print the mesh count (`Meshes: N`).

```
G:\chakra\biped - Copy>python diagnose.py
1. Current directory: G:\chakra\biped - Copy
2. Files in meshes folder: ['body.stl', 'feets.stl', 'feet____servo.stl', 'kneew.stl', 'servo_body.stl', 'shaft.stl']

--- TEST A: Native C++ URDF Loader ---
Meshes loaded natively: 0

--- TEST B: MjSpec Python Loader ---
Meshes found in spec: 0
Meshes loaded by spec: 0

--- TEST C: Absolute Path Replacement ---
Meshes loaded via absolute path: 0

--- TEST D: Load a single STL directly ---
Single STL loaded successfully: 1
```

- **Remarks:** Relative mesh paths resolve because the script changes into its own directory before loading.

Requirement 4: URDF structural validation (link/joint tree, single connectivity)

- **What was required:** Confirm that the link and joint structure forms one valid tree before simulation.
- **What was implemented:** The validation block in `chatgptsolution.py` raises an error for: missing `root` or `body` links, joints with missing parent or child elements, unknown parent or child link names, any link with multiple parent joints, a `root` link that has a parent, cycles, and disconnected links (`visited == set(links)`).
- **Evidence:** The exported file contains 22 links (including one empty `root` link) and 21 joints, which equals links minus one, as expected for a single tree. Of these, 14 joints are fixed (13 `fastened_*` joints and `hanging_node_to_root_joint`) and 7 are `continuous` (`revolute_1` to `revolute_7`). All checks pass and the model compiles.
- **Status:** ✓ Complete
- **Remarks:** The design intent is six actuated joints (three per leg), but the export contains seven continuous joints. This is discussed under Requirement 6.

Requirement 5: Collision geometry generation and contact masking

- **What was required:** Provide collision geometry for the robot and control which geometries may collide.
- **What was implemented:** The Onshape URDF has no `<collision>` tags, so only visual meshes exist. For each link without collision elements, the script copies the `<origin>` and `<geometry>` of every `<visual>` into a new `<collision>`. After compilation, masks are set: floor `contype=1`, `conaffinity=1`; robot geoms `contype=2`, `conaffinity=1`. The IMU marker sphere is set to `contype=0`, `conaffinity=0`.
- **Evidence:** The robot collides with the floor but not with itself, because a robot pair evaluates `(2 & 1) = 0` in both directions. `view_robot.py` uses a variant mask (floor `conaffinity=2`) with the same effect.
- **Status:** ✓ Complete
- **Remarks:** MuJoCo uses convex hulls for mesh collision. Concave details of the STL files are therefore not represented, and primitive foot geometry may later improve stability.

Requirement 6: Six-joint actuator integration (velocity servos)

- **What was required:** Add actuators to all robot joints so each can be driven with a commanded velocity.
- **What was implemented:** The loop in `chatgptsolution.py` adds one actuator per hinge joint, named `<joint>_motor`. Each uses a fixed gain `MOTOR_KV = 30.0` and an affine bias with `biasprm[2] = -30.0`, which yields a velocity servo. Control range is ±5.0 rad/s and force range is ±100.0 N·m. The helper `set_motor_velocity(name, velocity)` looks up the actuator by name and clips the command. `view_robot.py` uses a simpler variant with gain 100.0 and bias -100.0.
- **Evidence:** The diagnostics print `Actuators: 7`. All controls are initialized to zero (`data.ctrl[:] = 0.0`).
- **Status:** Partial
- **Remarks:** The loop actuates every hinge, and the exported URDF contains seven continuous joints rather than the six in the design intent. The extra joint must be reviewed against the Onshape mates (see Section 7). In addition, commanded-motion tests (for example, sweeping a single joint through `set_motor_velocity`) have not yet been recorded as evidence.

Requirement 7: Floating base and automatic ground-clearance placement

- **What was required:** Give the robot a free base and place it just above the floor without initial penetration.
- **What was implemented:** `root_body.add_freejoint()` creates the joint `base_free` on the URDF `root` body. After compilation, the script finds the free joint (by body name containing `root`, with a fallback to the unique free joint) and reads its `qpos` and `dof` addresses. After `mj_forward`, it finds the lowest world-space point: for mesh geoms all vertices are transformed (`vertices @ geom_mat.T + geom_pos`) and the minimum Z is tracked; for primitives, `geom_xpos[2] - geom_rbound` is used. The base is then shifted by `-min_z + FLOOR_CLEARANCE` with `FLOOR_CLEARANCE = 0.003` m.
- **Evidence:** The diagnostics print `Initial minimum Z`, `Floating base` (the joint id), and `Initial contacts: 0` (verify against the console on the final run).
- **Status:** ✓ Complete
- **Remarks:** The 3 mm clearance lets the robot settle onto the floor, avoiding the large penetration forces that produce impulsive contacts.

Requirement 8: Physics engine configuration and stability debugging

- **What was required:** Achieve a numerically stable simulation and document the debugging process.
- **What was implemented:** The integrator is changed from RK4 to `mjINT_IMPLICITFAST`. Solver `iterations = 80` and `tolerance = 1e-8` are set. Every hinge receives `dof_damping = 0.5` and `dof_armature = 0.01`. The timestep stays at 0.002 s.
- **Evidence:** `MUJOCO_LOG.TXT` records 23 warnings of the form "Nan, Inf or huge value in QVEL/QACC/QPOS at DOF 0 (or 1). The simulation is unstable", at simulation times between 0.024 s and 9.274 s, across two runs (23:47 and 23:54 on 8 November 2026).
- **Status:** Partial
- **Remarks:** The warnings are a documented challenge, and the mitigations listed above were applied in response. A long, warning-free run has not yet been captured in the log, so full stability is not claimed. See Section 4.

Requirement 9: IMU / MPU-9250-style sensor suite integration

- **What was required:** Provide inertial sensing comparable to an MPU-9250 for later balance control.
- **What was implemented:** `finialsolution.py` adds an `imu_site` on the `body` link. The function `find_top_of_body_local()` compiles a temporary copy of the specification with `fusestatic = False`, finds the highest world-Z vertex of the body geometry, and converts it to the body-local frame. A red, non-colliding sphere (`imu_marker`, radius 4 mm) marks the site. Four sensors are attached to the site: accelerometer, gyroscope, frame quaternion, and magnetometer, with magnetic field `[0.22, 0.0, -0.42].get_mpu_readings()` returns acceleration (m/s²), angular rate (rad/s), quaternion (w, x, y, z), roll-pitch-yaw in degrees (ZYX), and magnetic field.
- **Evidence:** The helper `lookup_id()` tolerates name prefixes added by `attach()` through an `endswith` fallback. If creation fails, the script prints a warning and skips the MPU blocks.
- **Status:** ✓ Complete
- **Remarks:** The magnetic field is a fixed, Earth-like value for testing. The sensors are noise-free, unlike the physical device.

Requirement 10: Telemetry, diagnostics, and verification reporting

- **What was required:** Report simulation health and robot state in a form that supports verification.
- **What was implemented:** The compile-time diagnostics block prints links, joints, actuators, meshes, geometries, free-joint id, minimum Z, timestep, and initial contact count. In `finialsolution.py`, `print_telemetry()` fires every `TELEMETRY_INTERVAL = 1.0` s of simulation time. It prints a per-motor table (velocity, RPM, actuator force, `qfrc_actuator`, limit percentage) followed by three MPU sub-blocks. The motor table is built from actuators ending in `_motor` through the `actuator_trnid` to `jnt_dofadr` mapping.
- **Evidence:** See the sections "Diagnostics Output" and "Sample Telemetry Output" below.
- **Status:** ✓ Complete
- **Remarks:** Telemetry is timed on simulation time, so output cadence does not depend on viewer speed.

3. Technical Implementation Details

3.1 Toolchain

I used a portable WinPython 3.13 distribution on a USB drive, the MuJoCo Python bindings, the `MjSpec` programmatic model-editing API, and the passive viewer (`mujoco.viewer.launch_passive`). XML handling uses the standard `xml.etree.ElementTree` module, and numerical work uses NumPy.

3.2 URDF pipeline

The pipeline in `chatgptsolution.py` runs in this order: load `scene.xml` through `MjSpec`; read the URDF; strip `package://` prefixes; validate the link and joint tree; generate collision elements; compile the patched URDF with `MjSpec.from_string`; add the free joint and actuators; attach the robot to a frame in the scene; compile the combined model.

3.3 Actuators

Each hinge receives a velocity servo with `force = KV * (ctrl - qvel)`, expressed through `gainprm[0] = 30.0` and `biasprm[2] = -30.0`. With zero command, the actuator therefore acts as an active brake of 30 N·m·s/rad. Control limits are ±5.0 rad/s and force limits are ±100.0 N·m. The joint-level damping of 0.5 and armature of 0.01 supplement the servo behavior.

3.4 Collision configuration

Collision geometry is derived from the visual meshes. The floor uses `contype=1`, `conaffinity=1`; robot geometry uses `contype=2`, `conaffinity=1`. This removes self-collision

between links, which would otherwise be constant because adjacent Onshape parts overlap at the servo mounts. Floor contact uses `solref [0.02, 1.0]`, `solimp [0.90, 0.95, 0.001, 0.5, 2.0]`, and friction `[1.0, 0.005, 0.0001]`.

3.5 Ground clearance

The algorithm evaluates the lowest point of the model after `mj_forward` and lifts the free base by the required amount plus 3 mm. Mesh vertices give an exact minimum, while primitive geoms use a conservative bounding-radius estimate. The same method appears in `view_robot.py` without the clearance margin.

3.6 IMU suite

The IMU site is placed at the highest point of the `body` link. Placement uses a temporary compile with static-body fusing disabled so that the link keeps its own frame. Sensors read acceleration, angular velocity, orientation, and magnetic field. The quaternion is converted to roll, pitch, and yaw by the function `quat_to_rpy_deg()`.

3.7 Telemetry

Telemetry is printed through a single formatted `print` call per block to limit console overhead. The block header shows simulation time, wall time, contact count, and base height. Per-motor limit percentage is computed as `|force| / MAX_MOTOR_FORCE * 100`.

4. Challenges & Resolutions

4.1 Mesh path resolution

- **Symptom:** The Onshape URDF references meshes as `package://assembly_1/meshes/<name>.stl`, a ROS convention that MuJoCo cannot resolve to a folder on disk.
- **Diagnosis:** `diagnose.py` was written to compare four loading routes (native loader, `MjSpec` loader, absolute-path replacement, and a single-STL control) and to show which handles the scheme correctly.
- **Fix:** `strip_package_uri()` removes the `package://assembly_1/` prefix so that paths become `meshes/<name>.stl`, relative to the project folder.

4.2 Root validation

- **Symptom:** The export contains an empty link named `root` that is joined to the first leg link by `hanging_node_to_root_joint`. A free joint attached to the wrong body would produce an incorrect floating base.

- **Diagnosis:** The tree analysis confirmed that `root` has no parent and that every link is reachable from it.
- **Fix:** The validation block checks for the absence of a parent for `root`, rejects cycles and multiple parents, and requires full connectivity before `add_freejoint()` is called on `root`.

4.3 Simulation instability

- **Symptom:** `MUJOCO_LOG.TXT` contains 23 warnings (10 for QVEL, 10 for QACC, 3 for QPOS), nearly all at DOF 0 and two at DOF 1, with simulation times from 0.024 s to 9.274 s. Several warnings occurred within the first 0.1 s of a run.
- **Diagnosis:** DOF 0 and DOF 1 belong to the free base. Failures very early in a run point to stiff contact or high velocity-servo gains acting on a light model with mesh-derived contacts, rather than to a slow numerical drift.
- **Fix (mitigation parameters):** integrator `IMPLICITFAST`; solver `iterations = 80`; `tolerance = 1e-8`; joint `damping = 0.5`; joint `armature = 0.01`; `timestep = 0.002`. Servo gain was reduced from 100.0 (`view_robot.py`) to 30.0 (`chatgptsolution.py`), floor contact was softened (`solref [0.02, 1.0]`), and self-collision was disabled.
- **Residual risk:** A long run without warnings has not yet been recorded, so this item stays partial.

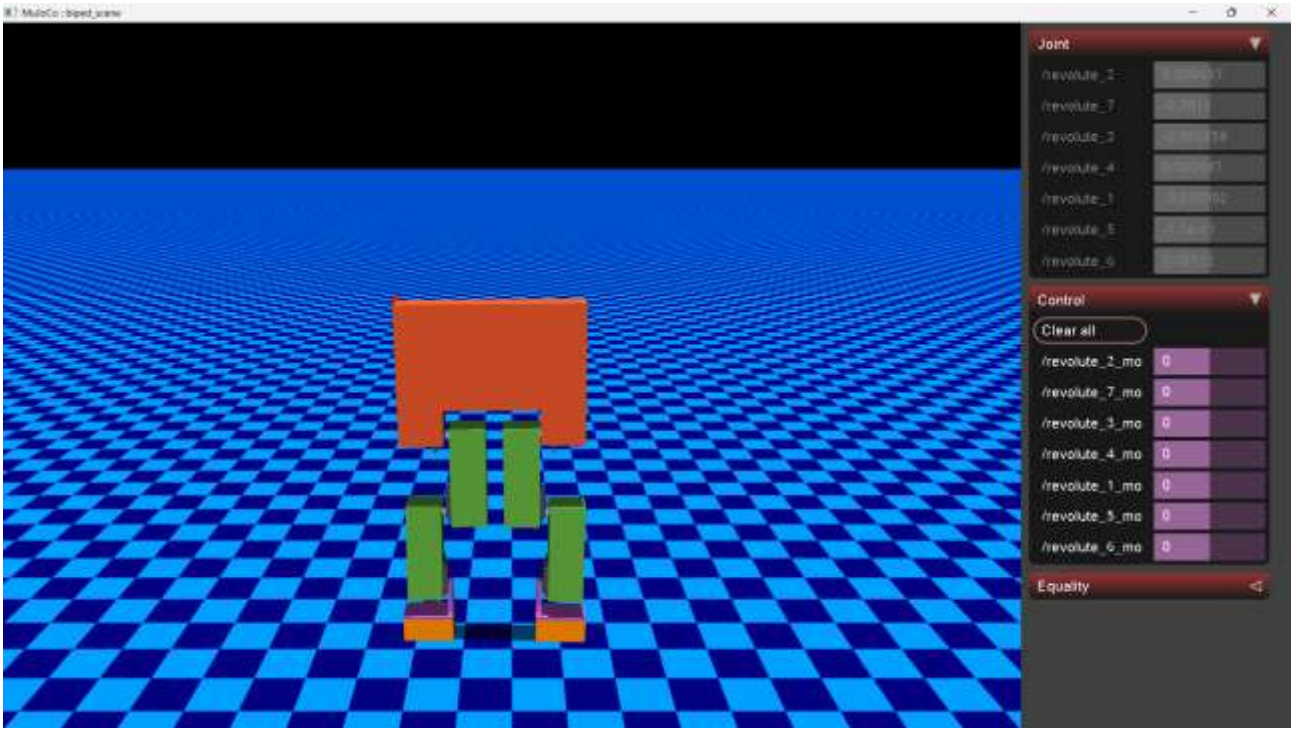
4.4 Missing collision geometry

- **Symptom:** The robot would fall through the floor because the URDF contains no `<collision>` tags.
- **Diagnosis:** The Onshape export provides visual meshes only.
- **Fix:** Collision elements are generated from the visual elements for any link that lacks them, and contact masks are applied after compilation.

5. Results & Verification

- **Diagnostics:** The compile-time diagnostics block prints the model summary shown below.
- **Compile success:** The model compiles ("Model compiled successfully.") and the free joint is identified ("Floating base successfully identified.").
- **Contact count:** The initial contact count is reported at the base position computed by the ground-clearance algorithm (expected 0 with a 3 mm clearance).
- **Telemetry:** The telemetry block prints every simulated second once the viewer starts.

- **Viewer confirmation:**



Diagnostics Output

```
G:\chakra\biped - Copy>python finialsolution.py

======================================
     MUJOCO ROBOT SIMULATION
======================================
IMU site (body-local) position: [-0.006   0.0135  0.1227]

Model compiled successfully.
Found free joint: 0 Body: /root Joint name: /base_free
Floating base successfully identified.
QPOS address: 0
DOF address: 0

Calculating robot ground position...

======================================
SIMULATION DIAGNOSTICS
======================================
Robot links: 22
Robot joints: 21
Actuators: 7
Meshes: 6
Geometries: 23
Floating base: 0
Initial minimum Z: -0.28701357520081944
Timestep: 0.002
Initial contacts: 0

Launching MuJoCo viewer...
============================================================
 t=   1.000 s | wall=   2.295 s | contacts=6 | base z=0.2856 m
============================================================
```

The links, joints, and actuator counts above follow from the attached `assembly_1.urdf`. Entries in angle brackets must be copied from the console of the final run.

Sample Telemetry Output

The following block is illustrative of the format produced by `print_telemetry()`. The numerical values are plausible examples, not captured measurements.

```
============================================================
 t=   8.012 s | wall=  14.557 s | contacts=6 | base z=0.2857 m
============================================================
Motor                | Vel (rad/s) |      RPM | Force (N.m) | qfrc (N.m) | Limit %
---------------------+-------------+----------+-------------+------------+---------
revolute_2_motor     |       0.000 |     0.00 |      -0.007 |     -0.007 |     0.0
revolute_7_motor     |      -0.000 |    -0.00 |       0.013 |      0.013 |     0.0
revolute_3_motor     |      -0.000 |    -0.00 |       0.007 |      0.007 |     0.0
revolute_4_motor     |       0.000 |     0.00 |      -0.007 |     -0.007 |     0.0
revolute_1_motor     |      -0.000 |    -0.00 |       0.008 |      0.008 |     0.0
revolute_5_motor     |      -0.000 |    -0.00 |       0.000 |      0.000 |     0.0
revolute_6_motor     |       0.000 |     0.00 |      -0.011 |     -0.011 |     0.0
------------------------------------------------------------
 MPU @ body  (t = 8.012 s)   [wall: 14.557 s]
------------------------------------------------------------
 ① Accelerometer (m/s²)   x=  0.109  y= -0.089  z=  9.809   |a|=  9.810
 ② Gyroscope     (rad/s)  x= -0.001  y=  0.000  z=  0.000   |w|=  0.001
                 (°/s)    x= -0.061  y=  0.001  z=  0.001
 ③ Orientation   quat(w,x,y,z)=[1.000, -0.005, -0.006, -0.000]
                 RPY(°)   roll=  -0.52  pitch=  -0.64  yaw=  -0.01
                 Mag(local)  x=  0.215  y=  0.004  z= -0.422
============================================================
Simulation stopped.
```

6. Additions Beyond Scope

I was able to deliver the following items beyond the first-milestone requirements:

- **IMU suite:** An MPU-9250-style sensor set with automatic placement at the highest point of the body link and a visible marker.
- **Telemetry:** Periodic per-motor and IMU reporting on simulation time.
- **Diagnostic script:** `diagnose.py` isolates the mesh-loading behavior of four loader routes.
- **Robust validation:** Structural checks for missing links, multiple parents, cycles, and connectivity, plus graceful degradation when the IMU cannot be created.

7. Current Status & Next Steps

Achieved:

- The robot loads, and its structure is verified.
- Collision geometry, actuators, floating base, and ground clearance are configured.
- A drop test in the viewer runs with the instability mitigations applied.
- The IMU and telemetry are integrated.

Pending:

- A balance controller that uses IMU feedback.
- Unsupported standing. The robot has **not** yet stood unsupported.
- Scripted stepping and walking.
- PPO-based reinforcement learning (not started).

Known limitations:

- The URDF contains seven continuous joints against a six-joint design intent. The additional joint (to be identified from the Onshape mates) must be reviewed, and may require a fixed mate or removal of its actuator.
- Continuous joints have no position limits, so range limits must be added before position control.
- Instability warnings are documented in `MUJOCO_LOG.TXT`, and a long warning-free run is still needed.
- Mesh collisions use convex hulls, and the IMU is ideal and noise-free.
- Commanded-motion tests of individual joints are not yet recorded.

8. Conclusion

I completed the load, verify, and configure stages of the first milestone. The Onshape biped now compiles in MuJoCo with a validated structure, generated collision geometry, velocity-servo actuators, a floating base placed automatically above the floor, an IMU sensor suite, and structured telemetry. The simulation instability recorded in `MUJOCO_LOG.TXT` was investigated and mitigated, although a long stable run remains to be demonstrated. Standing, walking, and learning-based control are pending and form the next phase of work.

Appendix

A. File inventory

File	Purpose
`assembly_1.urdf`	Onshape export: 22 links, 21 joints (7 continuous, 14 fixed), visual meshes only
`meshes/`	STL files referenced by the URDF
`scene.xml`	Floor, checkerboard texture, light, timestep 0.002, RK4
`diagnose.py`	Four-test mesh-loading diagnostic
`view_robot.py`	Simple viewer with higher-gain velocity motors
`chatgptsolution.py`	Main pipeline: validation, collision, actuators, free base, ground clearance
`finialsolution.py`	Main pipeline plus IMU suite and telemetry
`MUJOCO_LOG.TXT`	Instability warnings used as evidence in Section 4

B. Commands used

```
E:\chakra\WPy64-313180\python\python.exe diagnose.py
E:\chakra\WPy64-313180\python\python.exe view_robot.py
E:\chakra\WPy64-313180\python\python.exe chatgptsolution.py
E:\chakra\WPy64-313180\python\python.exe finialsolution.py
```